

Git / GitHub 가이드 (한 사이클)

Feature Branch Workflow · CAB302 Shapes OOP · [Atlassian](#) · [각주](#)

1→9가 한 사이클. **최초** = 처음·새 PC·새 계정·새 저장소일 때만 (끝났으면 건너뛰기).
Syntax = 문법 (<이름> placeholder) · **사용 예** = 실제 값 (origin, main, feature/... 등). origin/main은 고정 문구가 아니라 흔한 값이다.

[최초] 계정·Git·저장소 → config → clone
① 기본브랜치 맞춤 → ② feature → ③ commit → ④ push → ⑤ PR → ⑥ 피드백 → ⑦ merge → ⑧ 정리

1. 시작 준비 **최초**
2. 기본 브랜치 맞춤
3. feature
4. add / commit
5. push
6. PR · 리뷰
7. 피드백
8. merge
9. 정리

1. 시작 준비 — 계정 · Git · 저장소

대부분 **최초** URL 있고 clone까지 끝났으면 [2번](#)으로.

1-a. GitHub 계정 **최초**

어디서	할 일
github.com → Sign up	가입 · 이메일 인증
프로필 → Settings → Emails	커밋용 이메일 ^[1]
(선택) Profile	이름·사진

1-b. Git 설치 **최초**

Syntax	의미	사용 예
git-scm.com 설치	Windows에 Git	기본 옵션 OK
git --version	설치 확인	git version 2.x.x

IntelliJ: Settings → Version Control → Git · VS Code: Settings에서 git.path

옵션	의미
GitHub Desktop	GUI로 clone-commit-push

1-c. 저장소 확보 **최초** · clone보다 앞 ^[2]

① 수업/팀 저장소 있음

#	GitHub	의미
1	원본 페이지	수업 org 등
2	Fork	내 복사본
3	또는 Settings → Collaborators	초대 수락

② 새로 만든다

#	GitHub	의미
1-2	+ → <i>New repository</i> · 이름 · Public/Private	
3	"Add a README" 권장 [3]	첫 커밋
4-5	<i>Create</i> · <i>Collaborators</i> 초대	

1-d. 로컬 신원 **최초** · PC/계정

Syntax	의미	사용 예
git config --global user.name "<이름>"	커밋 이름 [1]	..."Alice Kim"
git config --global user.email "<이메일>"	Emails와 동일	..."alice@qut.edu.au"
git config --global -l	확인 (보조)	

옵션 / 보조	의미	사용 예
git config --global color.ui <모드>	터미널 색	... color.ui auto
git config --global core.excludesfile <파일>	PC 전역 ignore	[19]

1-e. 로컬에 저장소 두기 **최초** — 보통 clone / 옵션 init

권장: git clone

Syntax	의미	사용 예
Code → URL 복사	HTTPS/SSH	
git clone <URL>	원격→로컬 [4]	git clone https://github.com/나/repo.git
cd <폴더>	이동	cd shapes

이미 clone됐으면 재clone 말고 [2번](#) 로그인/토큰 [5]

옵션: git init — clone과 둘 다 하지 않음.

Syntax	의미	사용 예
cd · mkdir <프로젝트> · cd	프로젝트 폴더	mkdir example-repo
(선택) mkdir <소스폴더>	소스 폴더	mkdir src
git init	빈 저장소(.git) [4]	
파일 → add → commit	첫 로컬 커밋	4번과 동일
git remote add <원격> <URL>	원격 연결	git remote add origin https://...
git push -u <원격> <브랜치>	첫 업로드 + 추적	git push -u origin main

보조	의미	사용 예
git remote -v	원격 확인	
git remote add <원격> <원본URL>	Fork 원본 [6]	... upstream https://...
SSH keys + ssh-keygen	SSH [7]	최초 선택

1-f. 기본 브랜치 보호 **최초** · 팀장 권장

Settings → Branches / Rules → 기본 브랜치 직접 push 금지 [8]

1-g. .gitignore **최초** · 옵션

커밋하면 안 되는 패턴을 루트 .gitignore에 작성 [19]

예	의미
logs/ · *.class · .idea/	로그·빌드·IDE
git add .gitignore → commit	규칙도 버전 관리

1-h. 확인 최초 직후 · 또는 의심 시

확인 (명령)	기대 (예시)
git status · git log --oneline -5	On branch main · 최근 커밋
GitHub 페이지 · IDE Open	IntelliJ <i>Open</i> / VS Code <i>Open Folder</i>

2. 최신 기본 브랜치 맞추기 [매 사이클]

Syntax	의미	사용 예
git checkout <브랜치> / switch <브랜치>	브랜치 이동	git checkout main
git pull <원격> <브랜치>	원격 받아 병합 ^[9]	git pull origin main

기본 브랜치(보통 *main*) · Fork면 *Sync fork* 후 pull

보조	의미	사용 예 / 주의
git fetch <원격>	원격 정보만	git fetch origin
git merge <원격>/<브랜치>	fetch 후 병합	git merge origin/main
git reset --hard <원격>/<브랜치>	강제 일치 ^[10]	... origin/main · 미저장 삭제
git reset --hard <커밋>	지정 커밋으로 맞춤	공유 기본 브랜치 비권장

3. Feature branch [매 사이클]

Syntax	의미	사용 예
git checkout -b <이름>	새 갈래 ^[11]	git checkout -b feature/add-square

(선택) 웹 Create branch · 보통 로컬 후 5번 push
IntelliJ: 우측 하단 브랜치 → New Branch · **VS Code:** 좌측 하단 브랜치 → Create new branch...

보조 / 옵션	의미	사용 예
git branch / -a	로컬 / 원격 목록	
git branch <이름>	생성만 (이동 안 함)	이후 checkout
git checkout <이름>	기존 브랜치 이동	... feature/add-square

4. 수정 → Stage → Commit [매 사이클]

Syntax	의미	사용 예
(에디터 수정)	코드	Square.java
git status	변경 확인 ^[12]	
git add <파일>	stage	git add src/shapes/Square.java
git commit -m "<메시지>"	로컬 기록 ^[13]	..."Add Square class"

IntelliJ: Commit 창 체크=add, Commit, Git→Push/Update Project
VS Code: Source Control +=stage, Commit, ...→Push/Pull

보조 — stage/차이/로그	의미	사용 예
git add . · git diff · git diff --staged	전체 stage / 미·스테이지 차이	
git restore --staged <파일> · git reset <파일>	stage 취소	... Square.java
git log · --oneline · --follow <파일> · --stat -M	히스토리 / 리네임	
git log ..<>A> · git diff ...<A> · git show <SHA>	브랜치 비교 / 커밋 내용	main..feature/...

보조 — 삭제·이동	의미	사용 예
git rm <파일> · git mv <옛> <새>	삭제/이동 + stage	git rm Old.java

보조 — stash	의미
git stash · list · pop · drop	임시 저장 / 목록 / 적용 / 버리기

5. Feature Push [매 사이클]

Syntax	의미	사용 예
git push -u <원격> <브랜치>	원격 업로드 + 추적 [14]	git push -u origin feature/add-square
git push	(-u 이후) 현재 브랜치 → upstream	추가 커밋 후
git push <원격> <브랜치>	원격·브랜치 명시 [20]	git push origin main

Compare & pull request · Branches · 기본 브랜치는 아직 그대로

보조 — push vs push <원격> <브랜치>	git push	git push <원격> <브랜치>
대상	현재 브랜치 upstream	인자로 준 원격/브랜치
feature에서	그 feature 원격 (정상)	예: origin feature/... OK · origin main 은 보통 하지 않음
기본 브랜치에서	upstream이면 비슷	예: git push origin main

6. Pull Request · 리뷰 [매 사이클]

PR = GitHub 기능 [15]

#	GitHub	의미 / 예
1	Pull requests → New	요청
2	base ← compare	예: base=main ← compare= feature/...
3-4	제목 · Reviewers · Create	
5-6	Files changed / Review · Conversation	Approve 등

선택 CLI	의미	사용 예
gh pr create	CLI PR	gh pr create --title "..."
Checks / Actions	CI 있으면 통과 확인	

7. 피드백 반영 [필요 시]

Syntax	의미
수정 → add → commit → push	같은 PR에 반영

Conversation · Reply / Resolve

보조 (충돌)	의미	사용 예
checkout <기본> → pull → checkout <feature> → merge <기본>	최신 반영 [16]	main / feature/add-square
충돌 마커 정리 → add → commit → push	<<<<<< 제거	
PR Resolve conflicts	웹 해결	
git rebase <브랜치>	재배치 [17]	git rebase main

8. Merge [매 사이클]

GitHub	의미
Merge pull request → Confirm	feature→기본 브랜치 [18]
(선택) Delete branch	원격 feature 삭제

웹 대신 로컬 merge (보조):

```
git checkout <기본브랜치>      # 예: main
git pull
git pull <원격> <feature>      # 예: origin feature/add-square
git push
```

보조	의미	사용 예
git merge <브랜치>	지정 브랜치를 현재에 병합	git merge feature/add-square
git revert <커밋>	실수 취소 (새 커밋)	git revert 7e8263c

9. 정리 · 다음 사이클 [매 사이클]

Syntax	의미	사용 예
git checkout <기본브랜치>	공식 복귀	git checkout main
git pull <원격> <기본브랜치>	최신본	git pull origin main
git branch -d <feature>	로컬 feature 삭제	git branch -d feature/add-square

Delete branch · (선택) Releases / tag 예: v1.0

다음 기능 → 2번부터. 새 PC/새 저장소 → 1번부터. 저장소 생성 → 1-c (clone 1-e보다 앞).

각주

- 커밋 메타데이터. GitHub Emails에 같은 주소 필요. --global =이 PC 사용자 전체. [↑ 본문으로](#) [↑](#)
- 저장소 없으면 clone URL 없음. 과제는 Fork/Collaborator가 흔함. [↑ 본문으로](#)
- README 시 원격 첫 커밋. 빈 저장소면 로컬 첫 커밋 후 push. [↑ 본문으로](#)
- clone =원격 복사+히스토리+origin. init =현재 폴더에 빈 저장소만(원격은 나중에 remote add + push). 같은 폴더에 둘 다 쓰지 않음. [↑ 본문으로](#)
- HTTPS는 Token/Credential/SSH. IntelliJ-VS Code 브라우저/계정 로그인 가능. [↑ 본문으로](#)
- 흔한 별칭: origin=내 fork, upstream=원본. fetch upstream → merge → push origin main. [↑ 본문으로](#)
- SSH는 나중에 가능. 공개키만 GitHub. [↑ 본문으로](#)
- 기본 브랜치 실수 push 거절 → feature→PR만. [↑ 본문으로](#)
- pull ≈fetch+merge. 최신 기본 브랜치에서 feature 분기. [↑ 본문으로](#)
- reset --hard는 로컬 변경 삭제 가능. [↑ 본문으로](#)
- 예: feature/add-square. “main만 공식”은 팀 규칙. [↑ 본문으로](#)
- Untracked→Staged→Unmodified. Stage=이번 커밋만 고르기. [↑ 본문으로](#)
- Commit=로컬. GitHub는 push. [↑ 본문으로](#)

14. `-u` 이후 `git push` 만. feature push≠기본 브랜치 변경. [↑ 본문으로](#)
15. PR=병합 요청+리뷰. 중간 도움 요청에도. [↑ 본문으로](#)
16. 충돌=같은 줄 동시 수정. 자주 2번으로 예방. [↑ 본문으로](#)
17. Rebase=히스토리 재작성. 초심자는 merge. [↑ 본문으로](#)
18. Merge 후 기본 브랜치=새 확정본. Squash/Merge 팀 통일. [↑ 본문으로](#)
19. `.gitignore` =저장소별 무시. `core.excludesfile` =PC 전역. 이미 추적 중이면 ignore만으로 안 빠짐 → `git rm --cached`. [↑ 본문으로](#)
20. `git push` =현재 브랜치→upstream. `git push <원격> <브랜치>` =직접 지정. 예 `origin main`은 placeholder에 넣은 값. feature에서 `push origin main`은 기본 브랜치 직접 push — PR과 어긋날 수 있음. [↑ 본문으로](#)